

LAPORAN PRAKTIKUM

TEKNOLOGI IoT II

KOMUNIKASI ASINKRON DAN WEBSOCKETS & TOPOLOGI JARINGAN TERDESENTRALISASI (ESP-NOW DAN MESH)



Disusun Oleh:

Salma Anggun Velyza (25782082)
TRI 3C

Dosen Pengajar:

Dr. Ir. Septafiansyah Dwi Putra, S.T., M.T.

Agus Ambarwari, S.Pd., M.Kom.

TEKNOLOGI REKAYASA INTERNET
JURUSAN TEKNOLOGI INFORMASI
POLITEKNIK NEGERI LAMPUNG

2025/2026

MODUL 4: Komunikasi Asinkron dan WebSockets

1. Tujuan Praktikum

- Memahami keterbatasan *Web Server* HTTP sinkron (masalah *blocking* dan *page refresh*).
- Mampu menginstal dan menggunakan pustaka `ESPAsyncWebServer` untuk melayani banyak klien (*browser*) sekaligus.
- Memahami konsep protokol komunikasi persisten dan *Full-Duplex* menggunakan *WebSockets*.
- Mampu membuat arsitektur sinkronisasi dua arah antara input perangkat keras (tombol fisik) dan antarmuka web *real-time* tanpa interupsi *blocking* dari sensor yang lambat.

2. Alat dan Bahan

- NodeMCU ESP8266 (1 buah)
- Kabel Micro USB (1 buah)
- *Breadboard* (1 buah)
- LED 5mm (1 buah)
- Resistor 220 Ohm (1 buah)
- *Push Button* / *Tactile Switch* (1 buah)
- Resistor 10k Ohm (1 buah untuk *pull-down* eksternal)
- Sensor Suhu & Kelembapan DHT22 (1 buah)
- Kabel *Jumper* secukupnya

3. Langkah Kerja

1. Perakitan Rangkaian:

- Menghubungkan anoda LED ke pin **D6 (GPIO 12)** dan katodanya melalui resistor 220 Ohm ke **GND**.
- Menghubungkan pin data sensor DHT22 ke pin **D4 (GPIO 2)**, VCC ke **3V3**, dan GND ke **GND**.
- Memasang *push button* pada *breadboard*, menghubungkan satu sisi ke **3V3** dan sisi seberangnya ke pin **D2 (GPIO 4)** yang dilengkapi resistor *pull-down* 10k Ohm ke **GND**.

2. Pemrograman & Pengunggahan (*Upload*):

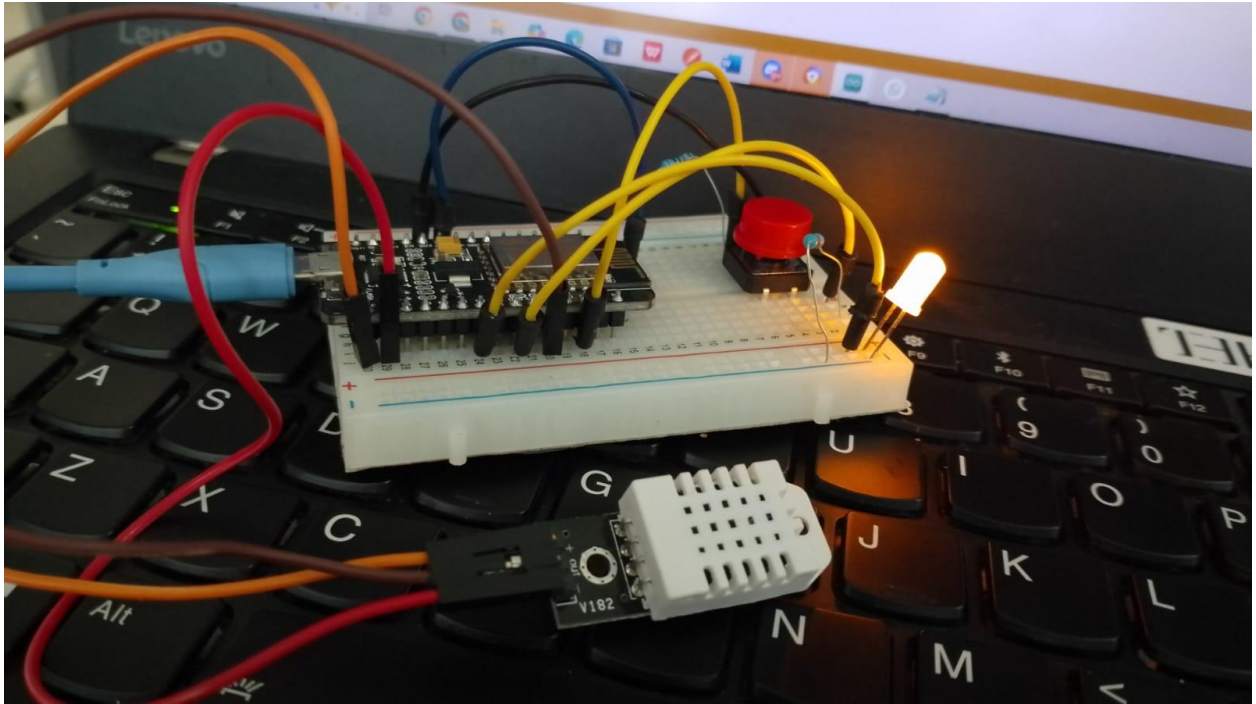
- Menyiapkan pustaka pendukung asinkron seperti `ESPAsyncWebServer` dan `ESPAsyncTCP`.
- Memasukkan kode program asinkron yang memadukan *Front-End* (HTML, CSS, JavaScript via WebSockets) dan *Back-End* (C++ dengan teknik *non-blocking* menggunakan `millis()`).
- Melakukan *compile* dan *upload* kode ke NodeMCU ESP8266.

3. Pengujian Sistem:

- Membuka alamat IP lokal NodeMCU di *browser* laptop dan *smartphone* secara berdampingan.
- Menguji kontrol tombol di antarmuka web untuk menyalakan/mematikan LED secara *real-time*.
- Menguji penekanan *push button* secara fisik di meja praktikum untuk melihat pembaruan status secara serentak di layar perangkat tanpa proses *refresh* halaman.

4. Hasil dan Pembahasan

- **Analisis WebSockets:** Berbeda dengan HTTP standar yang bersifat *request-response* (harus memuat ulang halaman/flickering), *WebSockets* membuka jalur komunikasi persisten dua arah (*full-duplex*). NodeMCU dapat secara proaktif mendorong (*push*) data telemetri berformat JSON ke *browser* seketika saat terjadi perubahan nilai suhu atau status tombol.
- **Non-Blocking dengan `millis()`:** Penggunaan `millis()` menggantikan fungsi `delay()`, sehingga CPU tidak mengalami *blocking* saat membaca sensor DHT22. Hal ini membuat respon kendali tombol fisik maupun web tetap instan tanpa jeda (*lag*).



```
sketch_oct5 | Arduino IDE 2.3.10
File Edit Sketch Tools Help
NodeMCU 1.0 (ESP-12E)
sketch_oct5.ino
1 #include <ESP8266WiFi.h>
2 #include <ESPAsyncTCP.h>
3 #include <ESPAsyncWebServer.h>
4 #include <DHT.h>
5
6 const char* ssid = "salma";
7 const char* password = "0987654321";
8
9 // Konfigurasi Pin
10 const byte dhtPin = 2; // D4 (GPIO 2)
11 const byte buttonPin = 4; // D2 (GPIO 4)
12 const byte ledPin = 12; // D6 (GPIO 12)
13
14 DHT dht(dhtPin, DHT22);
15
16 // Variabel Pelacak Status
17 bool ledState = false;
18 String currentTemp = "--";
19 int buttonState;
20 int lastButtonState = LOW;
21 unsigned long lastDebounceTime = 0;
22 const unsigned long debounceDelay = 50;
23 unsigned long lastTime = 0;
24
25 AsyncWebServer server(80);
26 AsyncWebSockets ws("/ws");
27
28 // HTML & JavaScript (Front-End)
29 const char index_html[] PROGMEM = R"rawliteral(
30 <!DOCTYPE html>
31 <html>
32 <head>
33 <meta name="viewport" content="width=device-width, initial-scale=1">
34 <title>Real Time IoT Web</title>
35 <style>
```

Output Serial Monitor X

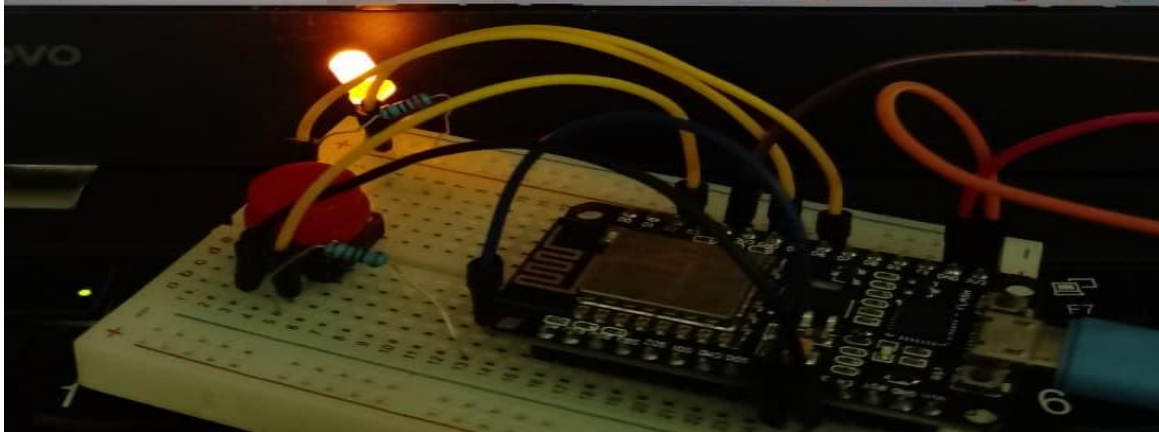
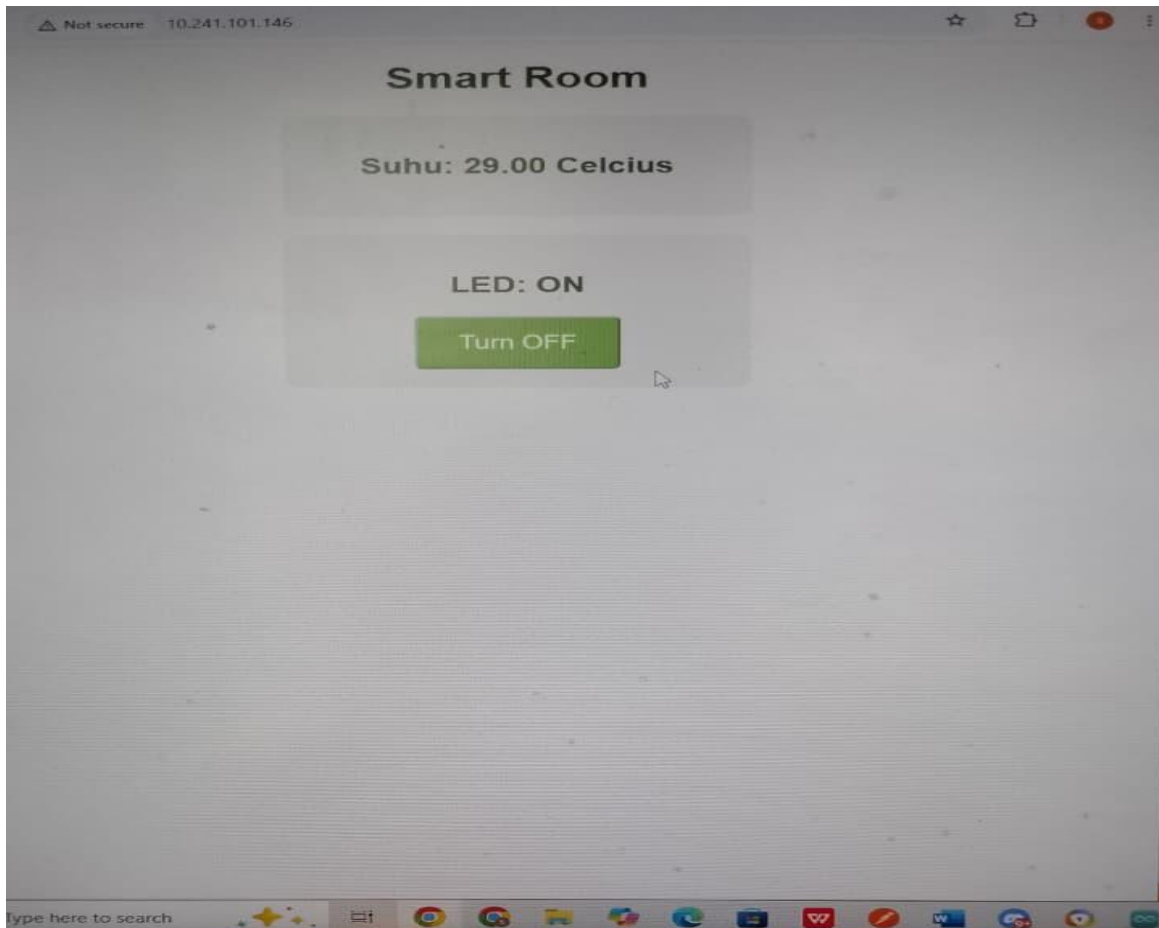
Message (Enter to send message to 'NodeMCU 1.0 (ESP-12E Module)' on 'COM4')

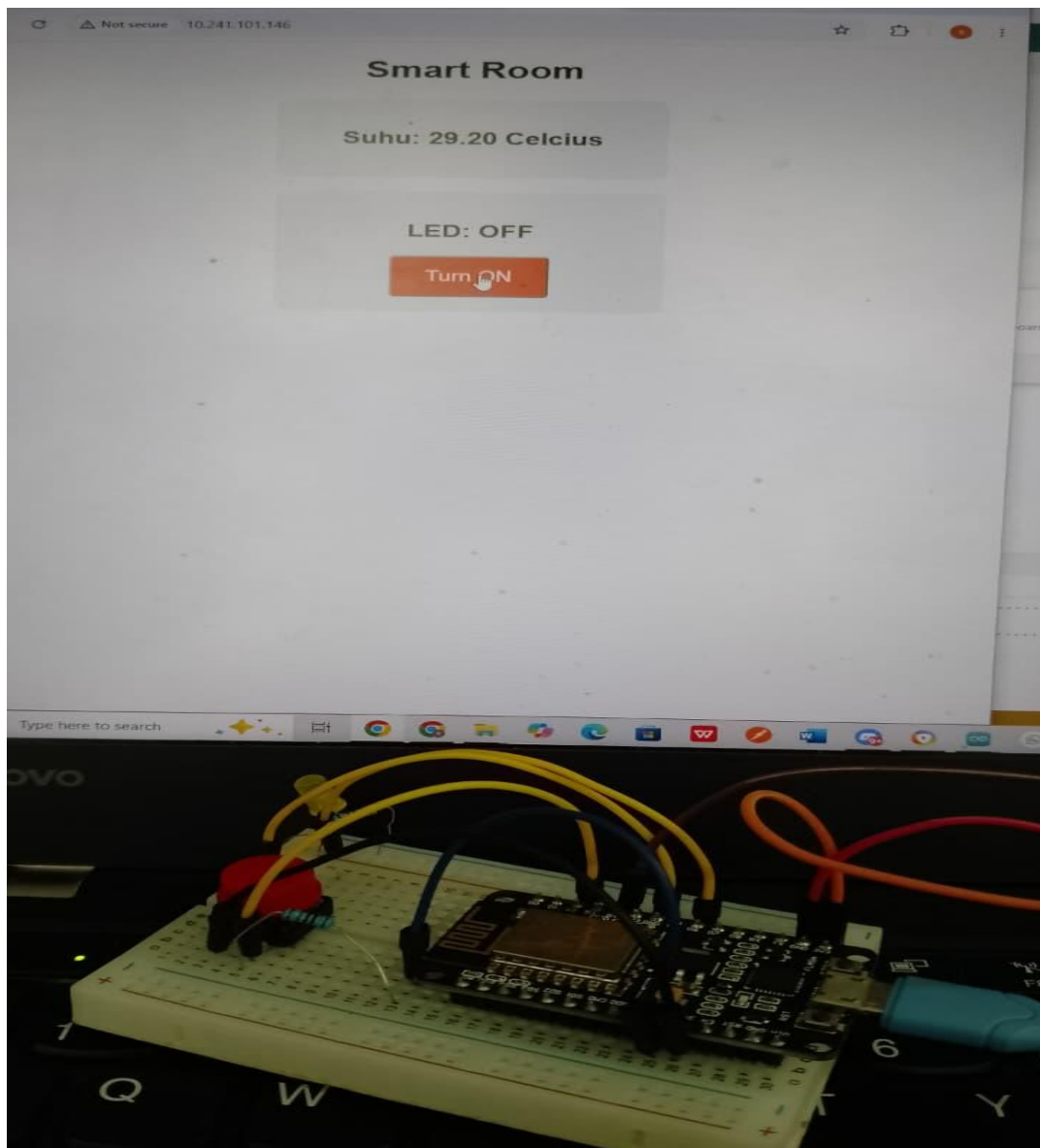
IP Address: 10.241.101.146

IP Address: 10.241.101.146

Ln 28, Col 33 NodeMCU 1.0 (ESP-12E Module) on COM4 4

32°C Cerah 4:17 PM 10/5/2025





Jawaban Latihan (Modul 4)

Latihan 1: *WebSockets* jauh lebih hemat *bandwidth* karena hanya mengirimkan paket data kecil berformat JSON secara asinkron lewat satu koneksi yang terus terbuka. Sementara metode *Meta Refresh* HTTP harus memuat ulang seluruh kerangka file HTML secara berulang-ulang yang boros data dan menyebabkan layar berkedip (*flickering*).

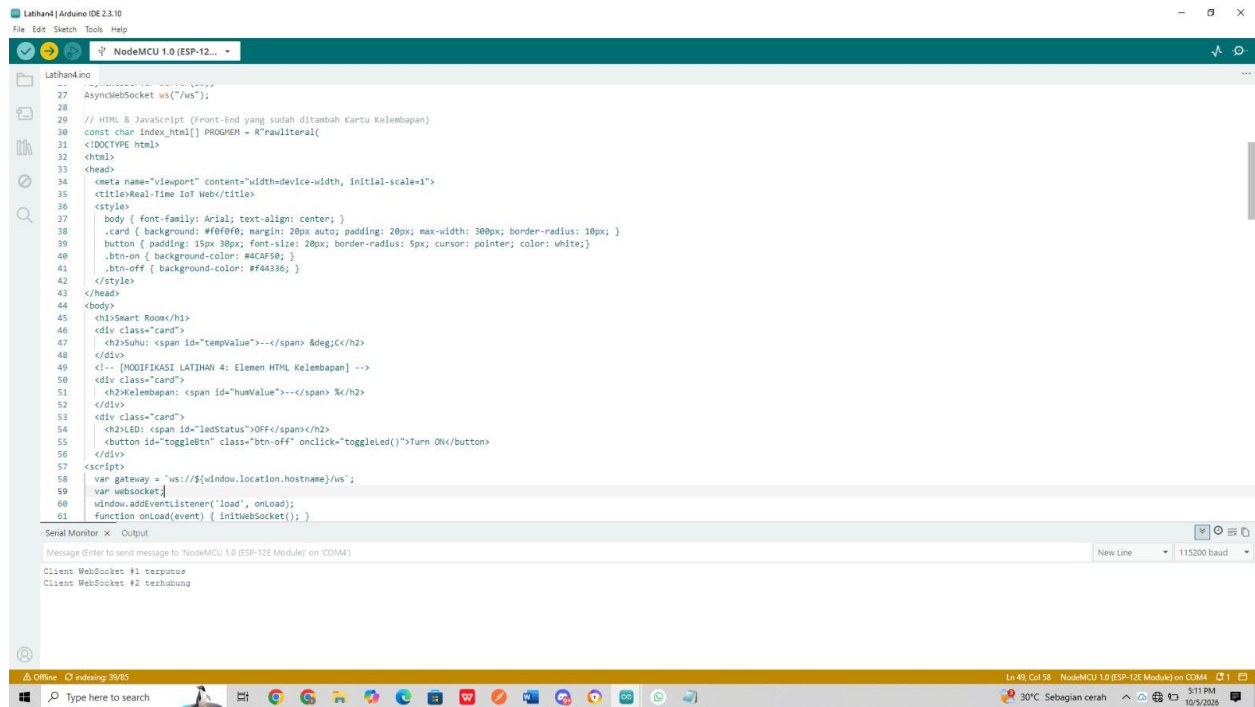
Latihan 2: Jika menggunakan `delay(3000)` di dalam `loop()`, CPU akan berhenti total (membeku) selama 3 detik. Akibatnya, sistem tidak akan responsif terhadap interaksi tombol fisik maupun koneksi WebSocket (*lag* parah atau terputus).

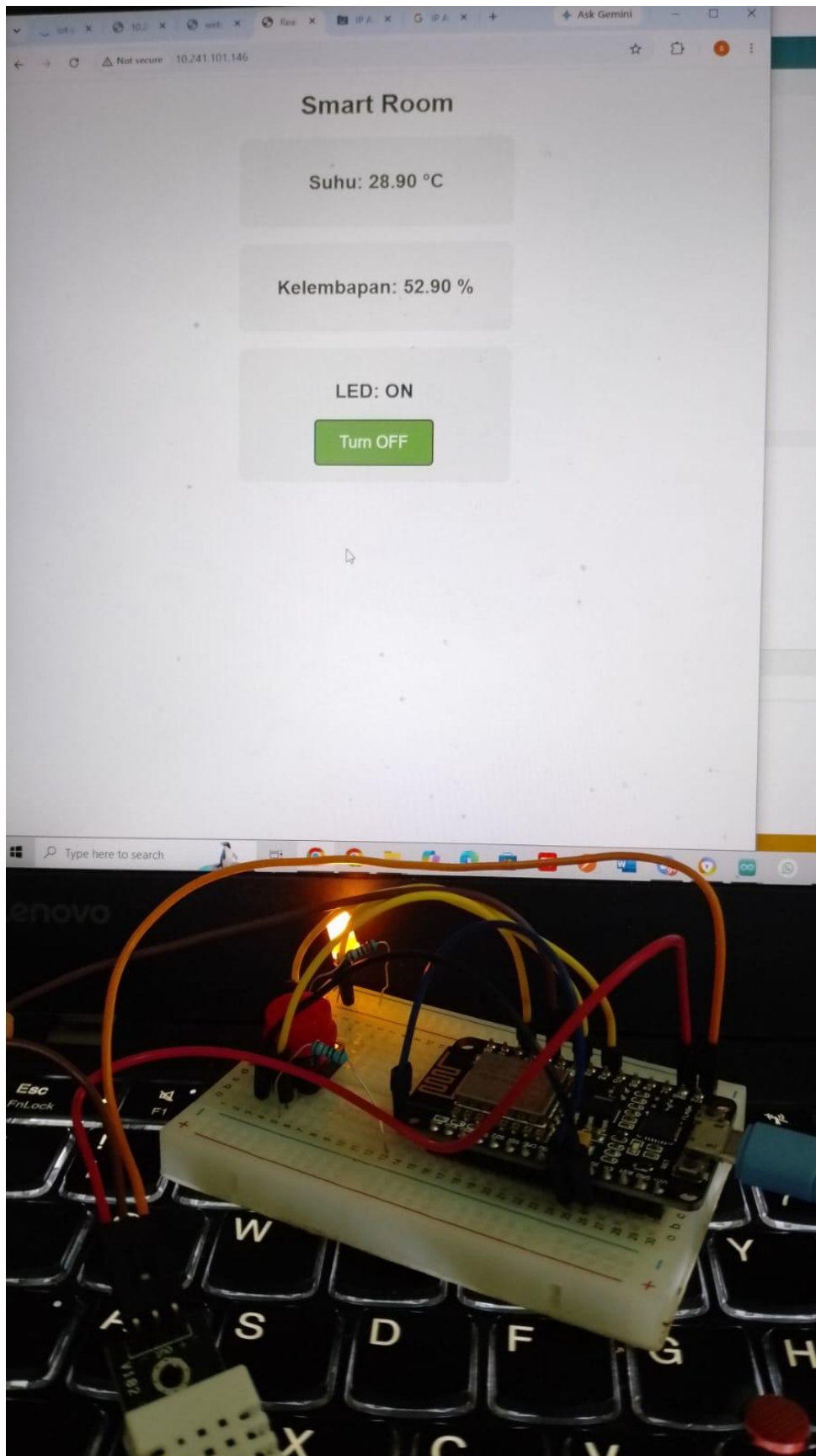
Latihan 3: Target elemen HTML yang menjadi target injeksi JavaScript meliputi:

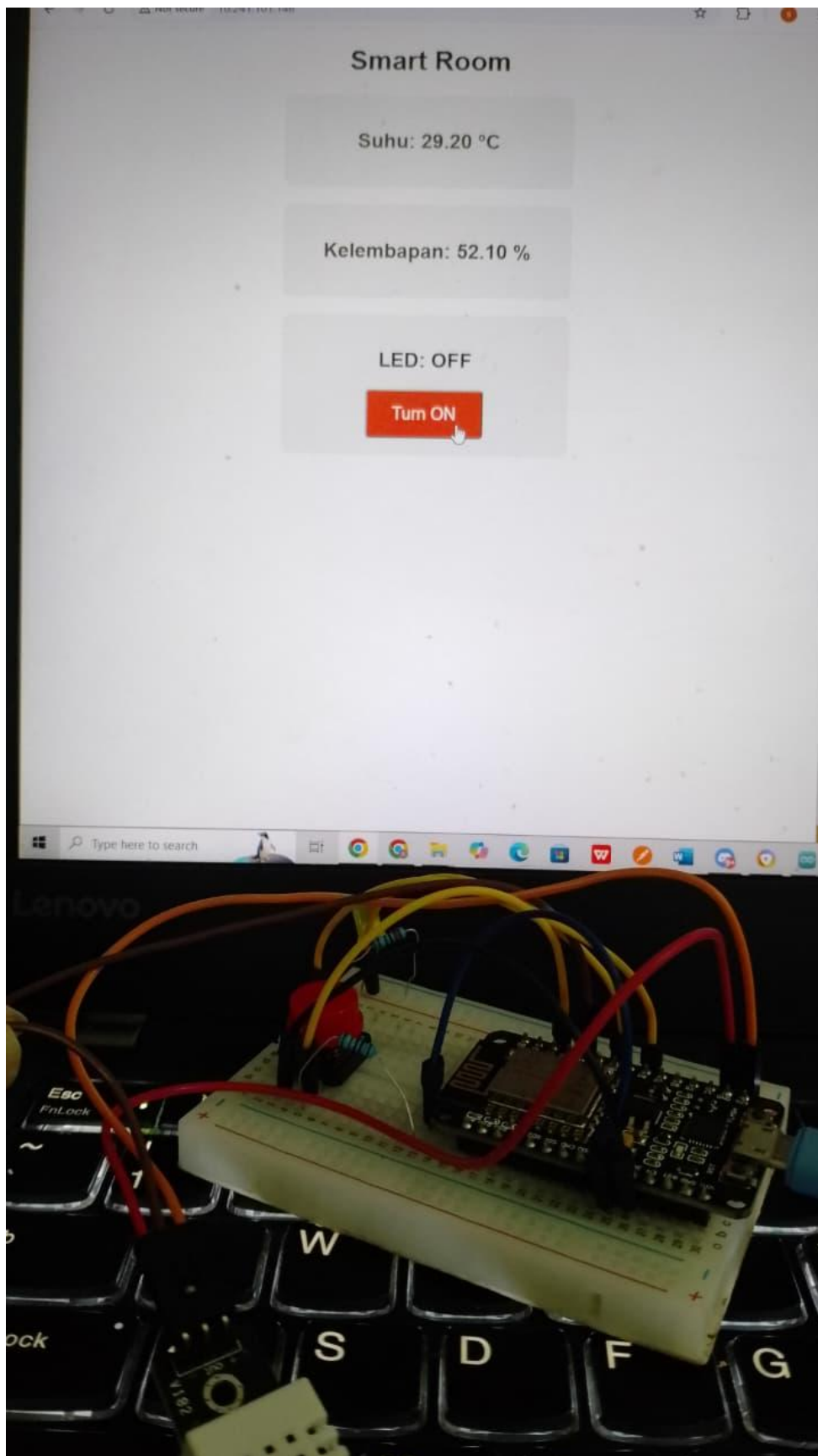
- `<span id="tempValue">` untuk menampilkan nilai suhu.
- `<span id="ledStatus">` untuk status teks ON/OFF.
- `<button id="toggleBtn">` untuk mengubah kelas gaya dan teks tombol (*Turn ON / Turn OFF*).

Latihan 4 (Modifikasi Kelembapan):

- *Back-End (C++)*: Menambahkan variabel `String currentHum = "--";`, membaca kelembapan via `dht.readHumidity()`, dan menyisipkannya ke dalam fungsi `notifyClients()`.
- *Front-End (HTML/JS)*: Menambahkan elemen kartu penampil `<div class="card"><h2>Kelembapan: <span id="humValue">--</span> %</h2></div>` serta memperbarui fungsi `onMessage()` untuk menangkap parameter kelembapan dari objek JSON.







Modul 5: Topologi Jaringan Terdesentralisasi (ESP-NOW dan Mesh)

1. Tujuan Pembelajaran

- Mahasiswa mampu menjelaskan perbedaan fundamental antara arsitektur jaringan berbasis infrastruktur (*Access Point/Router*) dan jaringan terdesentralisasi (*Peer-to-Peer*).
- Mahasiswa mampu membaca dan mendokumentasikan alamat fisik (*MAC Address*) dari antarmuka Wi-Fi NodeMCU ESP8266.
- Mahasiswa mampu mengonfigurasi dan memprogram komunikasi data searah dengan topologi *One-to-Many* berbasis protokol ESP-NOW.
- Mahasiswa mampu mengonfigurasi dan memprogram agregasi data dengan topologi *Many-to-One* berbasis protokol ESP-NOW.
- Mahasiswa mampu membangun jaringan *Self-Organizing Ad-Hoc Mesh* menggunakan pustaka `painlessMesh` tanpa ketergantungan konfigurasi *MAC Address* statis.

2. Dasar Teori

2.1 Jaringan Berbasis Infrastruktur vs Terdesentralisasi

Pada jaringan infrastruktur konvensional, setiap perangkat *Station* (STA) wajib terhubung ke *Access Point* pusat. Ketergantungan ini menimbulkan kerentanan berupa **Single Point of Failure** (titik kegagalan tunggal) serta latensi dan konsumsi daya tinggi akibat proses *handshake* autentikasi Wi-Fi. Sebaliknya, jaringan terdesentralisasi (*Peer-to-Peer* dan *Mesh*) membebaskan perangkat dari *router* sehingga komunikasi antar-node dapat berjalan secara mandiri dan langsung.

2.2 Protokol ESP-NOW

ESP-NOW adalah protokol komunikasi nirkabel berkecepatan tinggi tanpa koneksi (*connectionless*) buatan Espressif Systems yang berjalan langsung pada lapisan *MAC* (*Layer 2*) standar IEEE 802.11. Protokol ini meniadakan *overhead* lapisan jaringan TCP/IP, mendukung payload hingga 250 byte, dan sangat optimal dipadukan dengan mode *Deep Sleep* karena latensinya yang sangat rendah (< 5 ms).

2.3 Jaringan Ad-Hoc Mesh (`painlessMesh`)

Untuk mengatasi keterbatasan jarak jangkauan langsung (*single-hop* pada ESP-NOW), arsitektur *Mesh* memanfaatkan mekanisme *Multi-Hop Relay*. Setiap node bertindak secara simultan sebagai *soft-AP* dan *Station*, memungkinkan data diteruskan secara estafet melewati node perantara secara otonom (*Self-Organizing & Self-Healing*) tanpa perlu pencatatan alamat MAC statis.

3. Langkah Kerja & Implementasi Praktikum

3.1 Praktikum 1: Identifikasi MAC Address Perangkat NodeMCU

- **Tujuan** Membaca dan mencatat alamat fisik unik (*MAC Address* 48-bit) dari antarmuka Wi-Fi pada masing-masing *board* NodeMCU ESP8266 sebelum diintegrasikan ke dalam jaringan komunikasi ESP-NOW.
- **Dasar Teori & Penjelasan Kode:** Protokol komunikasi ESP-NOW tidak menggunakan alamat IP berbasis jaringan, melainkan langsung menggunakan pengalamatan perangkat keras fisik (*MAC Address*). Oleh karena itu, setiap *board* wajib diidentifikasi alamat MAC-nya terlebih dahulu.
 - `WiFi.mode(WIFI_STA)` ;: Menyiapkan modul radio Wi-Fi internal ke dalam mode *Station* agar alamat perangkat keras (*Hardware MAC*) siap dialokasikan oleh SDK.
 - `WiFi.disconnect()` ;: Memastikan mikrokontroler tidak mencoba melakukan proses penyambungan ke jaringan luar.
 - `WiFi.macAddress()` ;: Berfungsi mengambil string alamat fisik 48-bit bawaan pabrik yang bersifat unik secara global.
- **Kode Program Utama:**

```
#include <ESP8266WiFi.h>

void setup() {

  Serial.begin(115200);

  Serial.println();

  WiFi.mode(WIFI_STA);

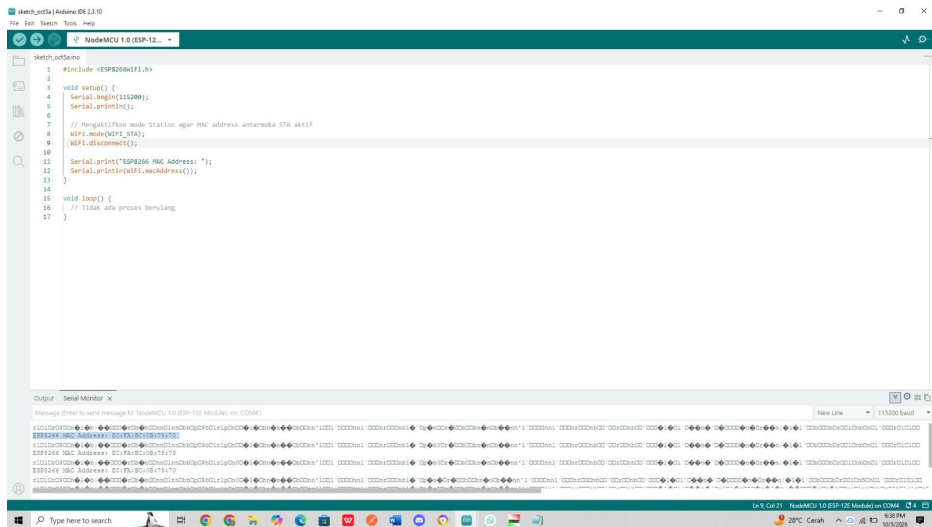
  WiFi.disconnect();


  Serial.print("ESP8266 MAC Address: ");

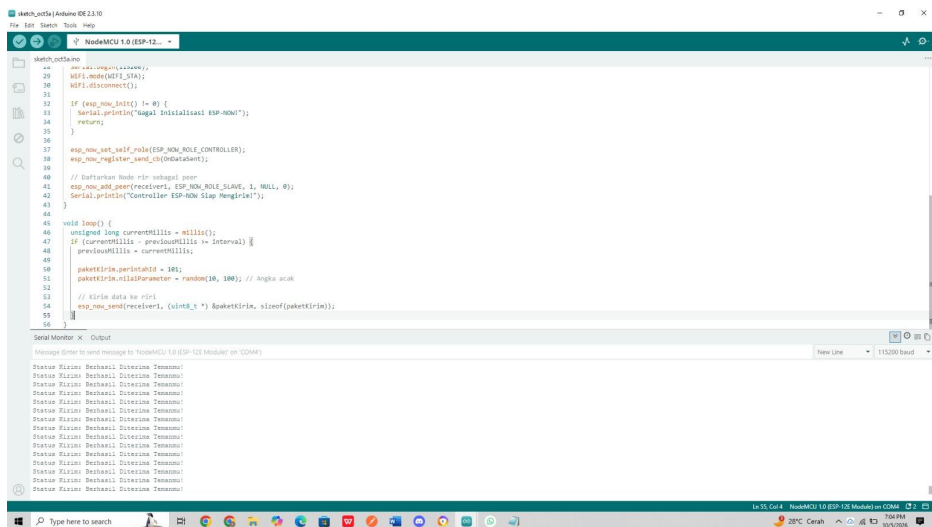
  Serial.println(WiFi.macAddress());

}

void loop() {}
```



```
1 #include <ESP8266WiFi.h>
2
3 void setup() {
4   Serial.begin(115200);
5   Serial.println();
6
7   // Mengaktifkan mode Station agar MAC address antarmuka STA aktif
8   WiFi.mode(WIFI_STA);
9   WiFi.disconnect();
10
11   Serial.print("ESP8266 MAC Address: ");
12   Serial.println(WiFi.macAddress());
13 }
14
15 void loop() {
16   // Tidak ada proses berulang
17 }
```



```
1 #include <ESP8266WiFi.h>
2
3 void setup() {
4   Serial.begin(115200);
5   Serial.println();
6
7   // Mengaktifkan mode Station agar MAC address antarmuka STA aktif
8   WiFi.mode(WIFI_STA);
9   WiFi.disconnect();
10
11   Serial.print("ESP8266 MAC Address: ");
12   Serial.println(WiFi.macAddress());
13 }
14
15 void loop() {
16   // Tidak ada proses berulang
17 }
```

Hasil Pengamatan & Analisis:

Analisis Tampilan Kode & Program:

- Pada fungsi `loop()`, mikrokontroler mengirimkan paket data secara berkala menggunakan fungsi `esp_now_send()` ke alamat MAC peer yang telah didaftarkan.
- Variabel muatan (*payload*) berisi perintahId = 101 dan nilaiParameter berupa angka acak (*random*).

Hasil Pengamatan pada Serial Monitor:

- Terlihat log pesan berulang bertuliskan **Status Kirim: Berhasil Diterima Temanmu!** (atau sesuai teks *callback* `OnDataSent` yang kamu atur di kode).
- Status pengiriman bernilai 0 (sukses), yang menandakan bahwa gelombang radio dari board Controller berhasil memancarkan data dan menerima sinyal balik *ACK* (*Acknowledgment*) fisik secara sukses dari board penerima (*Slave*).

3.2 Praktikum 2: Komunikasi Nirkabel One-to-Many (ESP-NOW)

- **Tujuan Praktikum:** Mengonfigurasi dan memprogram komunikasi data searah dengan topologi *One-to-Many* (Satu Pengirim ke Banyak Penerima) menggunakan protokol ESP-NOW secara langsung tanpa perantara *router*.
- **Dasar Teori & Penjelasan Alur:** Pada topologi ini, satu node bertindak sebagai *Controller (Sender)* untuk menyiarkan paket data instruksi secara serentak ke beberapa node *Slave (Receiver)*.
 - `esp_now_set_self_role(ESP_NOW_ROLE_CONTROLLER)` ;: Mengatur peran board sebagai pengirim paket.
 - `esp_now_add_peer(...)` : Mendaftarkan alamat fisik (*MAC Address*) perangkat penerima ke dalam tabel memori internal *Controller*.
 - `esp_now_send(0, ...)` ;: Parameter angka 0 digunakan agar paket data dipancarkan secara berurutan ke seluruh *peer* penerima yang telah terdaftar.

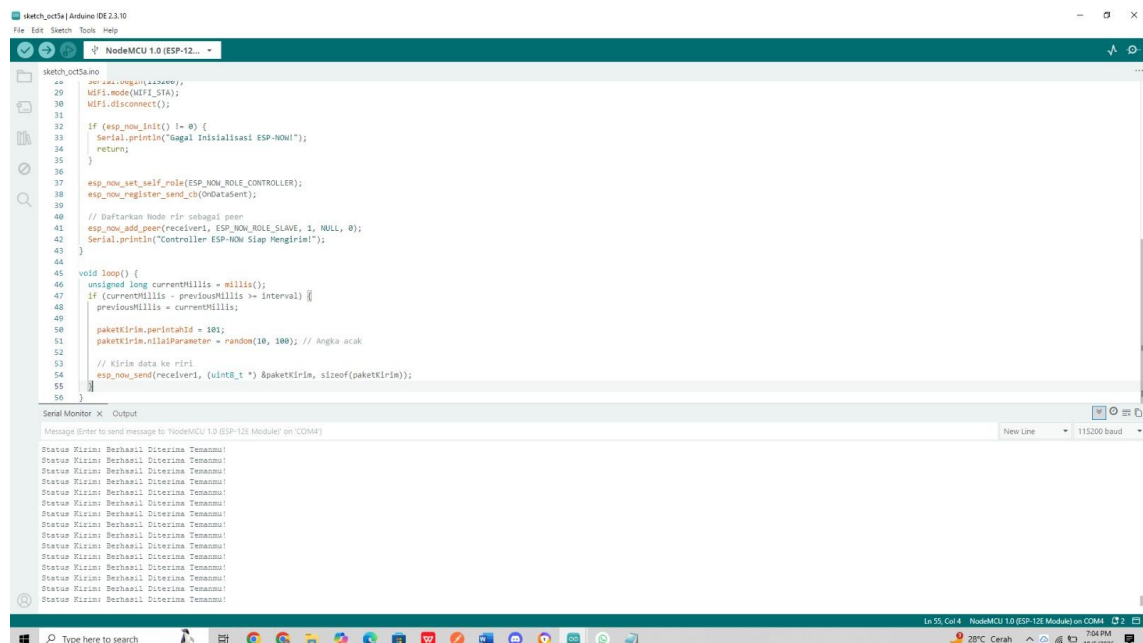
Hasil Percobaan & Pengamatan:

1. Tampilan Sisi Pengirim (Controller):

- Berdasarkan pengujian pada *Serial Monitor* pengirim, program sukses mencetak log berulang bertuliskan **Status Kirim: Berhasil Diterima Temanmu!**

Hal ini menunjukkan bahwa fungsi *callback* pengiriman (`OnDataSent`) mendeteksi nilai status pengiriman bernilai 0 (`sendStatus == 0`), yang berarti paket data berhasil dipancarkan dan dikonfirmasi oleh sinyal balik *ACK* dari board penerima.

Pengirim



The screenshot displays the Arduino IDE interface. The main window shows the sketch for the ESP-NOW sender. The code includes the following key sections:

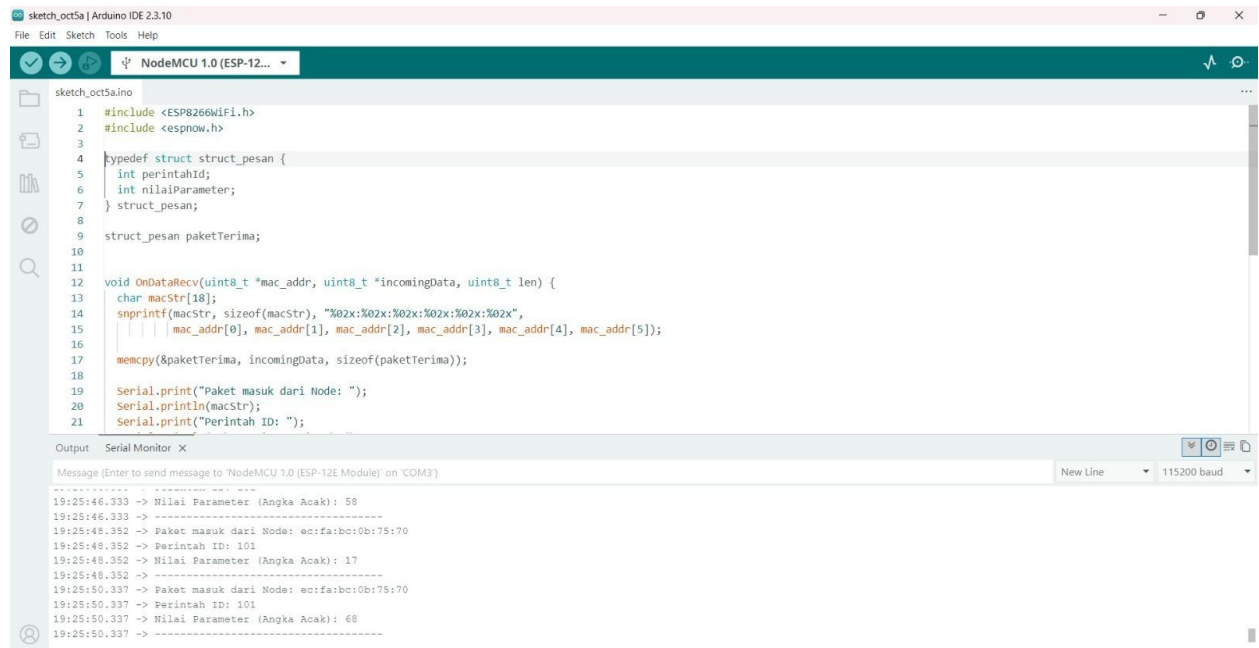
```
29 WiFi.mode(WIFI_STA);
30 WiFi.disconnect();
31
32 if (esp_now_init() != 0) {
33   Serial.println("Gagal Inisialisasi ESP-NOW!");
34   return;
35 }
36
37 esp_now_set_self_role(ESP_NOW_ROLE_CONTROLLER);
38 esp_now_register_send_cb(OnDataSent);
39
40 // Daftarkan node rpi sebagai peer
41 esp_now_add_peer(receiver1, ESP_NOW_ROLE_SLAVE, 1, NULL, 0);
42 Serial.println("Controller ESP-NOW Siap Mengirim!");
43 }
44
45 void loop() {
46   unsigned long currentMillis = millis();
47   if (currentMillis - previousMillis >= interval) {
48     previousMillis = currentMillis;
49
50     paketKirim.paketId = 101;
51     paketKirim.nilaiParameter = random(10, 100); // Angka acak
52
53     // Kirim data ke rpi
54     esp_now_send(receiver1, (uint8_t *) &paketKirim, sizeof(paketKirim));
55   }
56 }
```

The Serial Monitor at the bottom shows the output of the program, which consists of a continuous stream of the message: "Status Kirim: Berhasil Diterima Temanmu!". The baud rate is set to 115200.

Tampilan Sisi Penerima (Slave):

- Berdasarkan pengujian pada *Serial Monitor* penerima, data masuk secara berkala dari alamat MAC pengirim **ec:fa:bc:0b:75:70**.
- Variabel **Perintah ID: 101** beserta parameter angka acak (*random*) berhasil diterima dan disalin ke struktur data variabel penerima melalui fungsi `memcpy()` dengan akurat.

Penerima



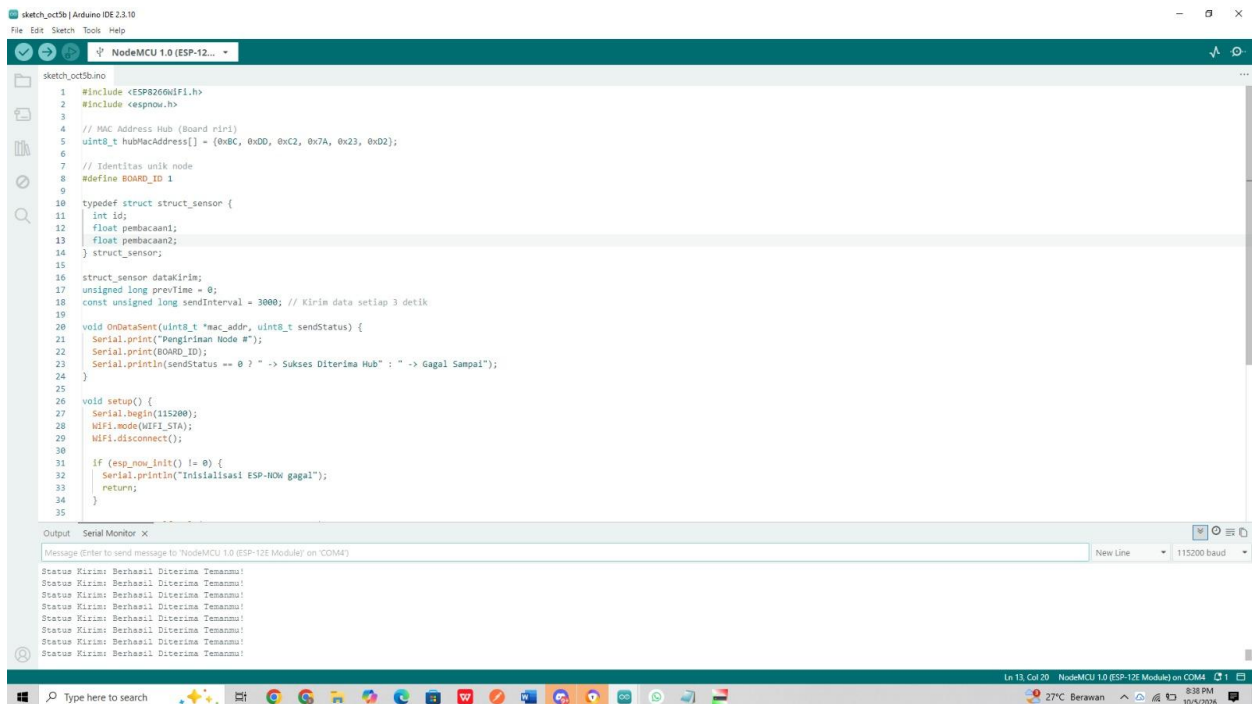
The screenshot shows the Arduino IDE interface with a sketch named 'sketch_oct5a.ino' for a NodeMCU 1.0 (ESP-12E) module. The sketch defines a structure for received data and implements the `onDataRecv` callback. The serial monitor shows the following output:

```
-----  
19:25:46.333 -> Nilai Parameter (Angka Acak): 58  
19:25:46.333 -> -----  
19:25:48.352 -> Paket masuk dari Node: ec:fa:bc:0b:75:70  
19:25:48.352 -> Perintah ID: 101  
19:25:48.352 -> Nilai Parameter (Angka Acak): 17  
19:25:48.352 -> -----  
19:25:50.337 -> Paket masuk dari Node: ec:fa:bc:0b:75:70  
19:25:50.337 -> Perintah ID: 101  
19:25:50.337 -> Nilai Parameter (Angka Acak): 68  
19:25:50.337 -> -----
```

Kesimpulan Praktikum: Protokol ESP-NOW terbukti mampu menjalankan komunikasi data nirkabel secara efisien langsung pada lapisan *Layer 2 MAC* tanpa memerlukan proses *handshake* ke router Wi-Fi luar.

Praktikum 3: Pengumpulan Data Telemetri Many-to-One (ESP-NOW)

- **Tujuan Praktikum:** Mengonfigurasi dan memprogram pengumpulan data dari beberapa node sensor di lapangan menuju satu node konsentrator pusat (*Aggregator Hub / Base Station*) menggunakan protokol ESP-NOW.
- **Dasar Teori & Penjelasan Alur:** Pada topologi *Many-to-One*, beberapa node pengirim (*Controller*) secara mandiri mengirimkan paket data pengukuran berkala ke satu alamat MAC tujuan yang sama, yaitu *Hub* pusat. Di sisi penerima pusat, data divalidasi menggunakan ID pengirim unik di dalam struktur data (*struct*) untuk mencegah kesalahan indeks memori (*array out-of-bounds*).



```
sketch_oct5b | Arduino IDE 2.3.10
File Edit Sketch Tools Help
NodeMCU 1.0 (ESP-12E...)
sketch_oct5b.ino
1 #include <ESP8266WiFi.h>
2 #include <espnow.h>
3
4 // MAC Address Hub (Board riri)
5 uint8_t hubMacAddress[] = {0x0C, 0x0D, 0xC2, 0x7A, 0x23, 0x02};
6
7 // Identitas unik node
8 #define BOARD_ID 1
9
10 typedef struct struct_sensor {
11   int id;
12   float pembacaan1;
13   float pembacaan2;
14 } struct_sensor;
15
16 struct_sensor dataKirim;
17 unsigned long prevTime = 0;
18 const unsigned long sendInterval = 3000; // Kirim data setiap 3 detik
19
20 void OnDataSent(uint8_t *mac_addr, uint8_t sendStatus) {
21   Serial.print("Pengiriman Node #");
22   Serial.print(BOARD_ID);
23   Serial.println(sendStatus == 0 ? " -> Sukses Diterima Hub" : " -> Gagal Sampai");
24 }
25
26 void setup() {
27   Serial.begin(115200);
28   WiFi.mode(WIFI_STA);
29   WiFi.disconnect();
30
31   if (esp_now_init() != 0) {
32     Serial.println("Inisialisasi ESP-NOW gagal");
33     return;
34   }
35
36   Output: Serial Monitor X
37   Message (Enter to send message to 'NodeMCU 1.0 (ESP-12E Module)' on 'COM4')
38   Status Kirim: Berhasil Diterima Tenanmu!
39   Status Kirim: Berhasil Diterima Tenanmu!
40   Status Kirim: Berhasil Diterima Tenanmu!
41   Status Kirim: Berhasil Diterima Tenanmu!
42   Status Kirim: Berhasil Diterima Tenanmu!
43   Status Kirim: Berhasil Diterima Tenanmu!
44   Status Kirim: Berhasil Diterima Tenanmu!
45   Status Kirim: Berhasil Diterima Tenanmu!
46   Status Kirim: Berhasil Diterima Tenanmu!
47   Status Kirim: Berhasil Diterima Tenanmu!
48   Status Kirim: Berhasil Diterima Tenanmu!
49   Status Kirim: Berhasil Diterima Tenanmu!
50   Status Kirim: Berhasil Diterima Tenanmu!
51   Status Kirim: Berhasil Diterima Tenanmu!
52   Status Kirim: Berhasil Diterima Tenanmu!
53   Status Kirim: Berhasil Diterima Tenanmu!
54   Status Kirim: Berhasil Diterima Tenanmu!
55   Status Kirim: Berhasil Diterima Tenanmu!
56   Status Kirim: Berhasil Diterima Tenanmu!
57   Status Kirim: Berhasil Diterima Tenanmu!
58   Status Kirim: Berhasil Diterima Tenanmu!
59   Status Kirim: Berhasil Diterima Tenanmu!
60   Status Kirim: Berhasil Diterima Tenanmu!
61   Status Kirim: Berhasil Diterima Tenanmu!
62   Status Kirim: Berhasil Diterima Tenanmu!
63   Status Kirim: Berhasil Diterima Tenanmu!
64   Status Kirim: Berhasil Diterima Tenanmu!
65   Status Kirim: Berhasil Diterima Tenanmu!
66   Status Kirim: Berhasil Diterima Tenanmu!
67   Status Kirim: Berhasil Diterima Tenanmu!
68   Status Kirim: Berhasil Diterima Tenanmu!
69   Status Kirim: Berhasil Diterima Tenanmu!
70   Status Kirim: Berhasil Diterima Tenanmu!
71   Status Kirim: Berhasil Diterima Tenanmu!
72   Status Kirim: Berhasil Diterima Tenanmu!
73   Status Kirim: Berhasil Diterima Tenanmu!
74   Status Kirim: Berhasil Diterima Tenanmu!
75   Status Kirim: Berhasil Diterima Tenanmu!
76   Status Kirim: Berhasil Diterima Tenanmu!
77   Status Kirim: Berhasil Diterima Tenanmu!
78   Status Kirim: Berhasil Diterima Tenanmu!
79   Status Kirim: Berhasil Diterima Tenanmu!
80   Status Kirim: Berhasil Diterima Tenanmu!
81   Status Kirim: Berhasil Diterima Tenanmu!
82   Status Kirim: Berhasil Diterima Tenanmu!
83   Status Kirim: Berhasil Diterima Tenanmu!
84   Status Kirim: Berhasil Diterima Tenanmu!
85   Status Kirim: Berhasil Diterima Tenanmu!
86   Status Kirim: Berhasil Diterima Tenanmu!
87   Status Kirim: Berhasil Diterima Tenanmu!
88   Status Kirim: Berhasil Diterima Tenanmu!
89   Status Kirim: Berhasil Diterima Tenanmu!
90   Status Kirim: Berhasil Diterima Tenanmu!
91   Status Kirim: Berhasil Diterima Tenanmu!
92   Status Kirim: Berhasil Diterima Tenanmu!
93   Status Kirim: Berhasil Diterima Tenanmu!
94   Status Kirim: Berhasil Diterima Tenanmu!
95   Status Kirim: Berhasil Diterima Tenanmu!
96   Status Kirim: Berhasil Diterima Tenanmu!
97   Status Kirim: Berhasil Diterima Tenanmu!
98   Status Kirim: Berhasil Diterima Tenanmu!
99   Status Kirim: Berhasil Diterima Tenanmu!
100  Status Kirim: Berhasil Diterima Tenanmu!
```

Kode Program Sisi Node Pengirim (Node 1):

```
#include <ESP8266WiFi.h>
#include <espnow.h>

// MAC Address HUB Penerima Pusat yang sudah disesuaikan dengan board-mu
uint8_t hubMacAddress[] = {0xEC, 0xFA, 0xBC, 0x0B, 0x75, 0x70};

// Identitas unik node (Ganti angka ini menjadi 2 untuk Node Pengirim kedua)
#define BOARD_ID 1

typedef struct struct_sensor {
  int id;
  float pembacaan1;
```

```

    float pembacaan2;
} struct_sensor;

struct_sensor dataKirim;
unsigned long prevTime = 0;
const unsigned long sendInterval = 3000; // Kirim data setiap 3 detik

void OnDataSent(uint8_t *mac_addr, uint8_t sendStatus) {
    Serial.print("Pengiriman Node #");
    Serial.print(BOARD_ID);
    Serial.println(sendStatus == 0 ? " -> Sukses Diterima Hub" : " -> Gagal
Sampai");
}

void setup() {
    Serial.begin(115200);
    WiFi.mode(WIFI_STA);
    WiFi.disconnect();

    if (esp_now_init() != 0) {
        Serial.println("Inisialisasi ESP-NOW gagal");
        return;
    }

    esp_now_set_self_role(ESP_NOW_ROLE_CONTROLLER);
    esp_now_register_send_cb(OnDataSent);

    // Daftarkan hub penerima tunggal
    esp_now_add_peer(hubMacAddress, ESP_NOW_ROLE_SLAVE, 1, NULL, 0);
}

void loop() {
    unsigned long now = millis();
    if (now - prevTime >= sendInterval) {
        prevTime = now;

        dataKirim.id = BOARD_ID;
        dataKirim.pembacaan1 = random(250, 350) / 10.0; // Simulasi suhu: 25.0 -
35.0 °C
        dataKirim.pembacaan2 = random(400, 800) / 10.0; // Simulasi kelembapan: 40.0
- 80.0 %

        esp_now_send(hubMacAddress, (uint8_t *) &dataKirim, sizeof(dataKirim));
    }
}

```

Kode Program Sisi Hub Konsentrator (Penerima Pusat):

```
#include <ESP8266WiFi.h>
#include <espnow.h>

typedef struct struct_sensor {
    int id;
    float pembacaan1;
    float pembacaan2;
} struct_sensor;

struct_sensor dataTerima;
// Array penampung data terakhir: Index 0 untuk Node 1, Index 1 untuk Node 2
struct_sensor records[2];

void OnDataRecv(uint8_t *mac_addr, uint8_t *incomingData, uint8_t len) {
    memcpy(&dataTerima, incomingData, sizeof(dataTerima));

    // Validasi ID agar tidak terjadi error index array out-of-bounds
    if (dataTerima.id >= 1 && dataTerima.id <= 2) {
        int index = dataTerima.id - 1;
        records[index].id = dataTerima.id;
        records[index].pembacaan1 = dataTerima.pembacaan1;
        records[index].pembacaan2 = dataTerima.pembacaan2;

        Serial.println("=====");
        Serial.printf("DATA TERBARU DARI NODE %d\n", dataTerima.id);
        Serial.printf("Parameter 1 (Suhu)      : %.2f °C\n",
records[index].pembacaan1);
        Serial.printf("Parameter 2 (Kelembapan): %.2f %%\n",
records[index].pembacaan2);
        Serial.println("=====");
    }
}

void setup() {
    Serial.begin(115200);
    WiFi.mode(WIFI_STA);
    WiFi.disconnect();

    if (esp_now_init() != 0) {
        Serial.println("Gagal Inisialisasi ESP-NOW Hub!");
        return;
    }
}
```

```

    esp_now_set_self_role(ESP_NOW_ROLE_SLAVE);
    esp_now_register_recv_cb(OnDataRecv);

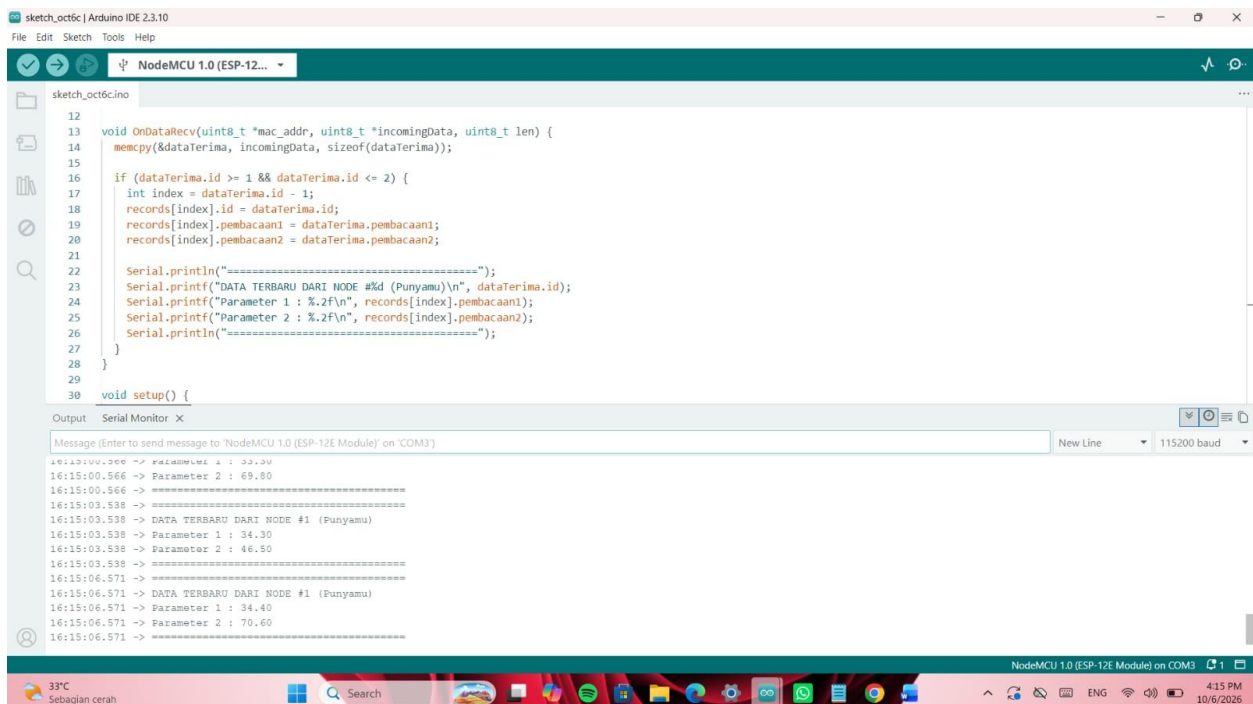
    Serial.println("Hub Konsentrator Siap Menerima Data Sensor...");
}

void loop() {
    // Loop bebas menangani tugas lain
}

```

Hasil Pengamatan pada Serial Monitor (Hub Konsentrator / Base Station):

2. Berdasarkan uji coba pengiriman dari beberapa perangkat, *Hub Konsentrator* berhasil menerima paket data sensor secara bersamaan dari node pengirim yang berbeda.
3. Layar *Serial Monitor* pada pusat konsentrator mencetak log data secara terstruktur dengan format blok sebagai berikut:



The screenshot shows the Arduino IDE interface. The top pane displays the sketch code for a NodeMCU 1.0 (ESP-12E) module. The code defines a function `OnDataRecv` that processes incoming data from multiple nodes, storing it in an array of records. It then prints the data to the serial monitor in a structured format. The bottom pane shows the Serial Monitor output, which displays the received data for two different nodes (Node #1 and Node #2) at different times, confirming that the hub can receive data from multiple sources simultaneously.

```

12
13 void OnDataRecv(uint8_t *mac_addr, uint8_t *incomingData, uint8_t len) {
14     memcpy(&dataTerima, incomingData, sizeof(dataTerima));
15
16     if (dataTerima.id >= 1 && dataTerima.id <= 2) {
17         int index = dataTerima.id - 1;
18         records[index].id = dataTerima.id;
19         records[index].pembacaan1 = dataTerima.pembacaan1;
20         records[index].pembacaan2 = dataTerima.pembacaan2;
21
22         Serial.println("=====");
23         Serial.printf("DATA TERBARU DARI NODE #%d (Punyamu)\n", dataTerima.id);
24         Serial.printf("Parameter 1 : %.2f\n", records[index].pembacaan1);
25         Serial.printf("Parameter 2 : %.2f\n", records[index].pembacaan2);
26         Serial.println("=====");
27     }
28 }
29
30 void setup() {

```

Serial Monitor Output:

```

16:13:00.000 -> Parameter 1 : 33.30
16:15:00.566 -> Parameter 2 : 69.80
16:15:00.566 -> =====
16:15:03.538 -> =====
16:15:03.538 -> DATA TERBARU DARI NODE #1 (Punyamu)
16:15:03.538 -> Parameter 1 : 34.30
16:15:03.538 -> Parameter 2 : 46.50
16:15:03.538 -> =====
16:15:06.571 -> =====
16:15:06.571 -> DATA TERBARU DARI NODE #1 (Punyamu)
16:15:06.571 -> Parameter 1 : 34.40
16:15:06.571 -> Parameter 2 : 70.60
16:15:06.571 -> =====

```

Analisis & Pembahasan:

1. Mekanisme Agregasi Many-to-One: Berbeda dengan topologi *One-to-Many* yang menyiarkan data ke banyak arah, pada skenario ini beberapa node pengirim (*Controller*) secara mandiri mengirimkan paket data pengukuran berkala ke satu alamat MAC tujuan yang sama yaitu *Hub* pusat.
2. Validasi ID dan Pencegahan Memory Corruption: Pada sisi kode program *Hub*, struktur data masuk divalidasi menggunakan nomor identifikasi unik (`BOARD_ID` atau `dataTerima.id`). Melalui blok pengondisian `if (dataTerima.id >= 1 && dataTerima.id <= 2)`, data langsung diarahkan ke indeks *array* penyimpanan yang tepat (`records[index]`). Mekanisme ini sangat krusial untuk mencegah terjadinya kesalahan indeks memori (*array out-of-bounds*) serta menghindari tercampurnya data telemetri antar-node yang masuk secara bersamaan.

Praktikum 4: Jaringan Sensor Ad-Hoc Multi-Hop (*painlessMesh*)

Tujuan Praktikum:

- Membangun jaringan *Self-Organizing Ad-Hoc Mesh* menggunakan pustaka *painlessMesh* tanpa ketergantungan konfigurasi *MAC Address* statis.
- Mengintegrasikan sensor fisik suhu & kelembapan (DHT11/DHT22) serta menyiarkan data telemetri berformat *JSON* secara otonom melintasi jaringan multi-hop.

Dasar Teori & Penjelasan Alur:

- Pada jaringan *Mesh*, setiap node ESP8266 bertindak secara simultan sebagai *Access Point* (soft-AP) dan *Station* (STA), memungkinkan data diteruskan secara estafet melewati node perantara (*multi-hop relay*).
- Pustaka *painlessMesh* membebaskan pengembang dari keharusan mencatat alamat MAC satu per satu; node cukup menyamakan nama jaringan (*MESH_PREFIX*) dan kata sandi bersama (*MESH_PASSWORD*).
- Penggunaan pustaka *ArduinoJson* memungkinkan pengemasan data telemetri menjadi string teks terstruktur, sedangkan *TaskScheduler* menggantikan fungsi `delay()` agar tugas perutean paket latar belakang tidak terganggu.

Kode Program Node-1 (Sensor Suhu & Kelembapan DHT22):

```
#include <painlessMesh.h>
#include <DHT.h>
#include <ArduinoJson.h>

#define MESH_PREFIX      "LabIoTMesh"           // Nama jaringan Mesh bersama
```

```

#define MESH_PASSWORD    "iotmeshpassword" // Kata sandi jaringan Mesh
#define MESH_PORT        5555              // Port komunikasi TCP Mesh

// Konfigurasi pin dan tipe sensor DHT
#define DHTPIN           2                  // Pin D4 (GPIO 2)
#define DHTTYPE          DHT22             // Sensor DHT22
DHT dht(DHTPIN, DHTTYPE);

const char *nodeName = "Node-1";

Scheduler userScheduler;
painlessMesh mesh;

void sendMessage();
void receivedCallback(uint32_t from, String &msg);
void newConnectionCallback(uint32_t nodeId);

Task taskSendMessage(TASK_SECOND * 5, TASK_FOREVER, &sendMessage);

void sendMessage() {
    float suhu = dht.readTemperature();
    float kelembapan = dht.readHumidity();

    if (isnan(suhu) || isnan(kelembapan)) {
        Serial.println("Gagal membaca sensor DHT!");
        return;
    }

    StaticJsonDocument<200> jsonBuffer;
    jsonBuffer["node"] = nodeName;
    jsonBuffer["suhu"] = suhu;
    jsonBuffer["kelembapan"] = kelembapan;

    String msg;
    serializeJson(jsonBuffer, msg);

    mesh.sendBroadcast(msg);
    Serial.println("Data Node 1 Terkirim: " + msg);
}

void receivedCallback(uint32_t from, String &msg) {
    Serial.printf("Pesan diterima dari node %u: %s\n", from, msg.c_str());
}

void newConnectionCallback(uint32_t nodeId) {

```

```

    Serial.printf("Koneksi baru terdeteksi, nodeId: %u\n", nodeId);
}

void setup() {
    Serial.begin(115200);
    dht.begin();

    mesh.setDebugMsgTypes(ERROR | STARTUP);
    mesh.init(MESH_PREFIX, MESH_PASSWORD, &userScheduler, MESH_PORT);
    mesh.onReceive(&receivedCallback);
    mesh.onNewConnection(&newConnectionCallback);

    userScheduler.addTask(taskSendMessage);
    taskSendMessage.enable();
}

void loop() {
    mesh.update();
}

```

Kode Program Node-2 (Sensor Cahaya LDR):

```

#include <painlessMesh.h>
#include <ArduinoJson.h>

#define MESH_PREFIX      "LabIoTMesh"
#define MESH_PASSWORD    "iotmeshpassword"
#define MESH_PORT        5555

#define LDR_PIN A0        // Pin Analog LDR pada NodeMCU

const char* nodeName = "Node-2";

Scheduler userScheduler;
painlessMesh mesh;

void sendMessage();
void receivedCallback(uint32_t from, String &msg);
void newConnectionCallback(uint32_t nodeId);

Task taskSendMessage(TASK_SECOND * 5, TASK_FOREVER, &sendMessage);

void sendMessage() {

```

```

int nilaiLDR = analogRead(LDR_PIN); // Membaca intensitas cahaya

DynamicJsonDocument jsonBuffer(1024);
jsonBuffer["node"] = nodeName;
jsonBuffer["cahaya"] = nilaiLDR;

String msg;
serializeJson(jsonBuffer, msg);

mesh.sendBroadcast(msg);
Serial.println("Data Node 2 Terkirim: " + msg);
}

void receivedCallback(uint32_t from, String &msg) {
    Serial.printf("Pesan diterima dari node %u: %s\n", from, msg.c_str());
}

void newConnectionCallback(uint32_t nodeId) {
    Serial.printf("Koneksi baru terdeteksi, nodeId: %u\n", nodeId);
}

void setup() {
    Serial.begin(115200);

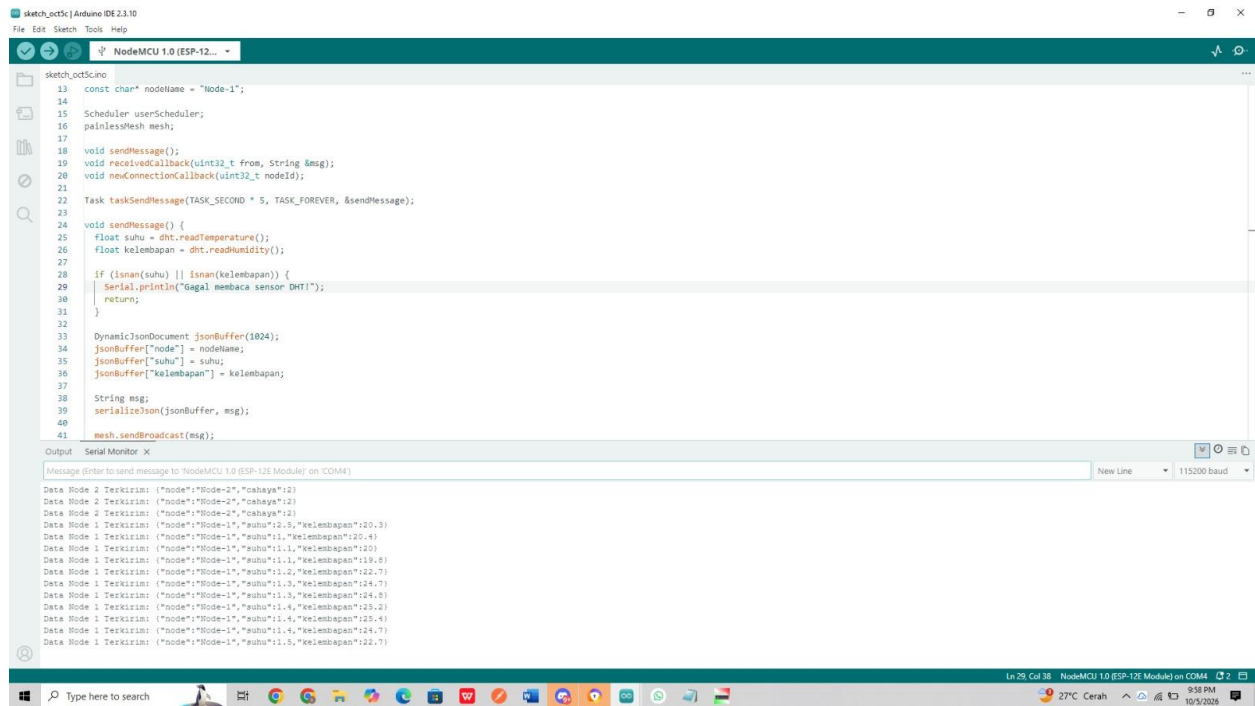
    mesh.setDebugMsgTypes(ERROR | STARTUP);
    mesh.init(MESH_PREFIX, MESH_PASSWORD, &userScheduler, MESH_PORT);
    mesh.onReceive(&receivedCallback);
    mesh.onNewConnection(&newConnectionCallback);

    userScheduler.addTask(taskSendMessage);
    taskSendMessage.enable();
}

void loop() {
    mesh.update();
}

```

1. Tangkapan Layar Serial Monitor Node-1:



```
sketch_oct5c.ino
13 const char* nodeName = "Node-1";
14
15 Scheduler userScheduler;
16 painlessMesh mesh;
17
18 void sendMessage();
19 void receivedCallback(uint32_t from, String &msg);
20 void newConnectionCallback(uint32_t nodeId);
21
22 Task taskSendMessage(TASK_SECOND * 5, TASK_FOREVER, &sendMessage);
23
24 void sendMessage() {
25   float suhu = dht.readTemperature();
26   float kelembapan = dht.readHumidity();
27
28   if (!isnan(suhu) || !isnan(kelembapan)) {
29     Serial.println("Gagal membaca sensor DHT!");
30     return;
31   }
32
33   DynamicJsonDocument jsonBuffer(1024);
34   jsonBuffer["node"] = nodeName;
35   jsonBuffer["suhu"] = suhu;
36   jsonBuffer["kelembapan"] = kelembapan;
37
38   String msg;
39   serializeJson(jsonBuffer, msg);
40
41   mesh.sendBroadcast(msg);
42 }
```

Output Serial Monitor X

Message (Enter to send message to 'NodeMCU 1.0 (ESP-12E Module)' on 'COM4')

Data Node 2 Terkirim: {"node":"Node-2","cahaya":12}

Data Node 2 Terkirim: {"node":"Node-2","cahaya":12}

Data Node 1 Terkirim: {"node":"Node-1","suhu":12.5,"kelembapan":120.3}

Data Node 1 Terkirim: {"node":"Node-1","suhu":11.1,"kelembapan":120.4}

Data Node 1 Terkirim: {"node":"Node-1","suhu":11.1,"kelembapan":119.8}

Data Node 1 Terkirim: {"node":"Node-1","suhu":11.2,"kelembapan":122.7}

Data Node 1 Terkirim: {"node":"Node-1","suhu":11.3,"kelembapan":124.7}

Data Node 1 Terkirim: {"node":"Node-1","suhu":11.3,"kelembapan":124.8}

Data Node 1 Terkirim: {"node":"Node-1","suhu":11.4,"kelembapan":125.2}

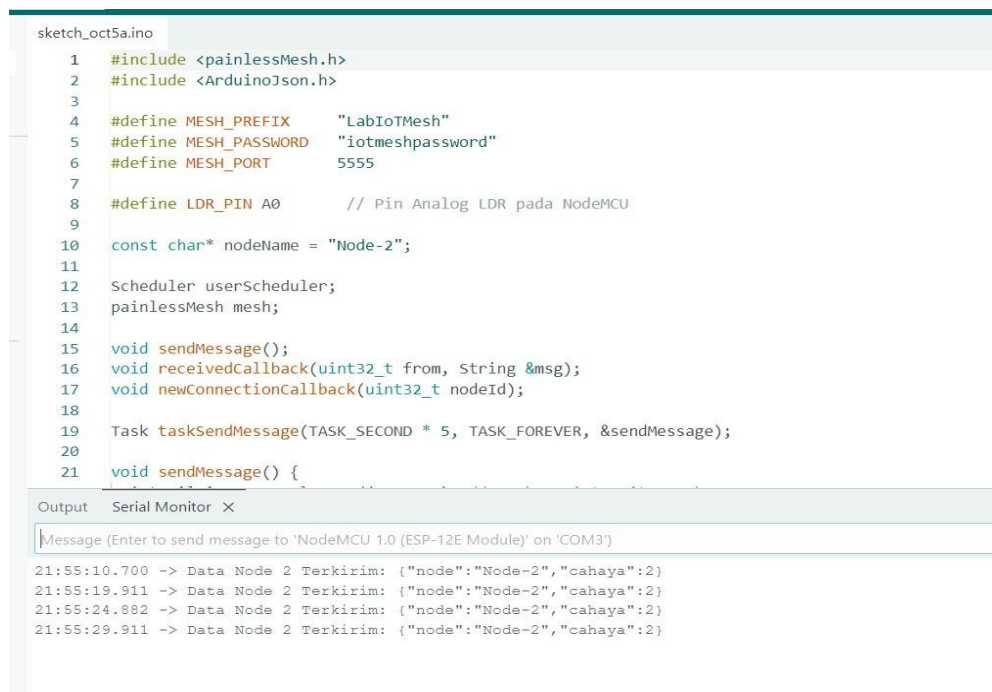
Data Node 1 Terkirim: {"node":"Node-1","suhu":11.4,"kelembapan":125.4}

Data Node 1 Terkirim: {"node":"Node-1","suhu":11.4,"kelembapan":124.7}

Data Node 1 Terkirim: {"node":"Node-1","suhu":11.5,"kelembapan":122.7}

Berdasarkan pengujian, *Serial Monitor* Node-1 sukses menyiarkan data telemetri lingkungan dengan format JSON secara berkala, contohnya {"node": "Node-1", "suhu": 21.1, "kelembapan": 20.4}. Node-1 juga berhasil menerima paket data yang diteruskan dari Node-2 di dalam jaringan yang sama.

2. Tangkapan Layar Serial Monitor Node-2:



```
sketch_oct5a.ino
1 #include <painlessMesh.h>
2 #include <ArduinoJson.h>
3
4 #define MESH_PREFIX "LabIoTMesh"
5 #define MESH_PASSWORD "iotmeshpassword"
6 #define MESH_PORT 5555
7
8 #define LDR_PIN A0 // Pin Analog LDR pada NodeMCU
9
10 const char* nodeName = "Node-2";
11
12 Scheduler userScheduler;
13 painlessMesh mesh;
14
15 void sendMessage();
16 void receivedCallback(uint32_t from, String &msg);
17 void newConnectionCallback(uint32_t nodeId);
18
19 Task taskSendMessage(TASK_SECOND * 5, TASK_FOREVER, &sendMessage);
20
21 void sendMessage() {
```

Output Serial Monitor X

Message (Enter to send message to 'NodeMCU 1.0 (ESP-12E Module)' on 'COM3')

21:55:10.700 -> Data Node 2 Terkirim: {"node":"Node-2","cahaya":2}

21:55:19.911 -> Data Node 2 Terkirim: {"node":"Node-2","cahaya":2}

21:55:24.882 -> Data Node 2 Terkirim: {"node":"Node-2","cahaya":2}

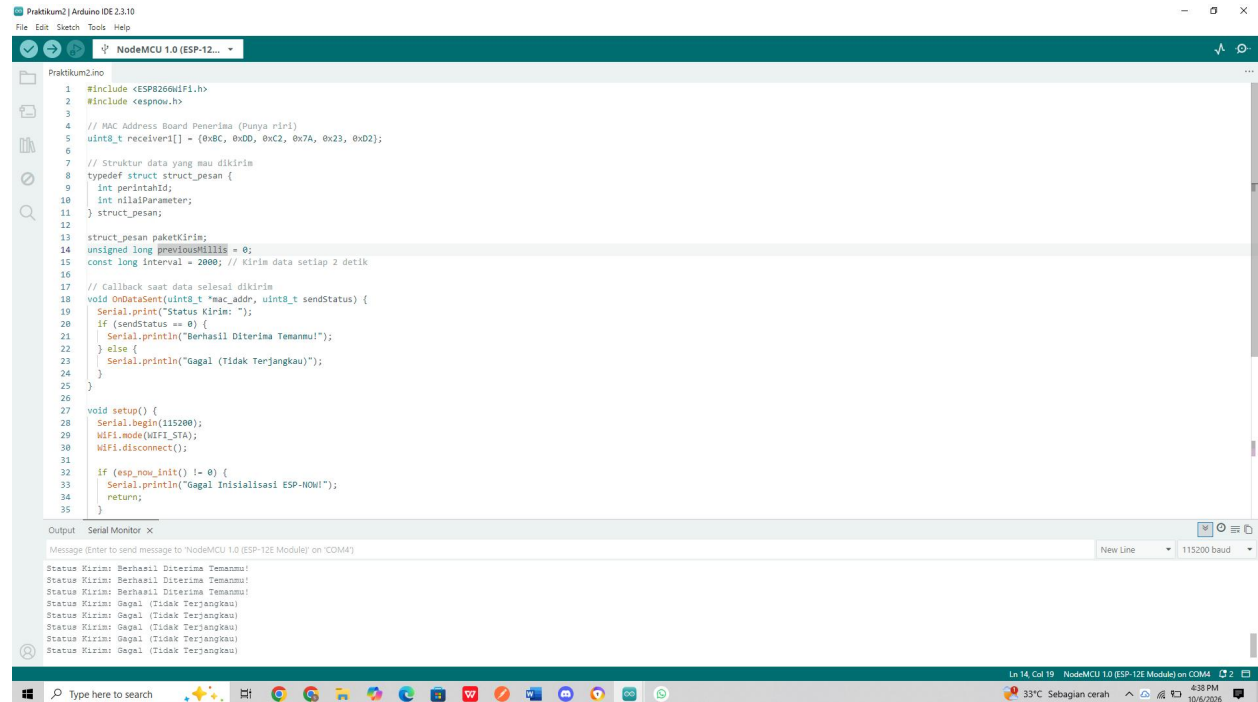
21:55:29.911 -> Data Node 2 Terkirim: {"node":"Node-2","cahaya":2}

Berdasarkan pengujian, *Serial Monitor* Node-2 sukses mengirimkan data pembacaan sensor cahaya LDR secara periodik dengan format JSON, contohnya {"node": "Node-2", "cahaya": 2}.

Analisis Pembentukan Topologi:

- Dengan menyamakan parameter `MESH_PREFIX` dan `MESH_PASSWORD`, kedua node terbukti dapat langsung mengenali satu sama lain secara otonom tanpa konfigurasi alamat MAC manual.
- Penggunaan pustaka `ArduinoJson` memudahkan proses pengemasan dan pertukaran data antar-node yang memiliki jenis sensor berbeda ke dalam satu standar teks terstruktur.

LATIHAN 1



The screenshot shows the Arduino IDE interface. The main window displays the code for 'Praktikum2.ino'. The code includes headers for ESP8266WiFi and ESPNow, defines a MAC address for the receiver, and sets up a structure for data packets. It includes a callback function 'OnDataSent' that checks the 'sendStatus' and prints messages to the Serial Monitor. The setup function initializes the serial port, WiFi, and ESPNow. The main loop calls 'OnDataSent' every 2000ms.

```
1 #include <ESP8266WiFi.h>
2 #include <ESPNow.h>
3
4 // MAC Address Board Penerima (Punya riri)
5 uint8_t receiver[] = {0x8C, 0x00, 0xC2, 0x7A, 0x23, 0x02};
6
7 // Struktur data yang mau dikirim
8 typedef struct struct_pesanan {
9   int perintahId;
10  int nilaiParameter;
11 } struct_pesanan;
12
13 struct_pesanan paketKirim;
14 unsigned long previousMillis = 0;
15 const long interval = 2000; // Kirim data setiap 2 detik
16
17 // Callback saat data selesai dikirim
18 void OnDataSent(uint8_t *mac_addr, uint8_t sendStatus) {
19   Serial.print("Status Kirim: ");
20   if (sendStatus == 0) {
21     Serial.println("Berhasil Diterima Temanmu!");
22   } else {
23     Serial.println("Gagal (Tidak Terjangkau)");
24   }
25 }
26
27 void setup() {
28   Serial.begin(115200);
29   WiFi.mode(WIFI_STA);
30   WiFi.disconnect();
31
32   if (esp_now_init() != 0) {
33     Serial.println("Gagal Inisialisasi ESP-NOW!");
34     return;
35   }
36 }
```

The Serial Monitor output shows the following messages:

```
Status Kirim: Berhasil Diterima Temanmu!
Status Kirim: Berhasil Diterima Temanmu!
Status Kirim: Berhasil Diterima Temanmu!
Status Kirim: Gagal (Tidak Terjangkau)
Status Kirim: Gagal (Tidak Terjangkau)
Status Kirim: Gagal (Tidak Terjangkau)
Status Kirim: Gagal (Tidak Terjangkau)
Status Kirim: Gagal (Tidak Terjangkau)
```

i & ii. Hasil Pengamatan saat Kabel Power Dicabut:

- **Observasi:** Ketika salah satu *board* penerima (misal NodeMCU 2) dicabut kabel USB-nya dari sumber daya, *Serial Monitor* pada *NodeMCU Controller* pengirim akan langsung mendeteksi perubahan status pengiriman.
- **Jawaban:** Saat salah satu *board* penerima dicabut dari sumber daya USB, *Serial Monitor* pada perangkat *Controller* pengirim secara otomatis mengubah status log pengirimannya dari "Berhasil Diterima" menjadi "Gagal (Tidak Terjangkau)" secara konsisten pada alamat MAC perangkat yang bersangkutan.

Pertanyaan iii (Mekanisme Lapisan Layer 2 MAC tanpa TCP):

- **Penjelasan:** Meskipun ESP-NOW berjalan secara *connectionless* tanpa protokol TCP/IP, lapisan perangkat keras MAC (IEEE 802.11 Layer 2) tetap menggunakan sistem konfirmasi perangkat keras berupa **sinyal ACK (*Acknowledgment*)**. Ketika *Controller* mengirim paket, chip radio penerima yang aktif wajib langsung membalas dengan sinyal *ACK*. Saat kabel daya dicabut, chip penerima mati sehingga tidak ada sinyal *ACK* yang dikembalikan, yang kemudian dibaca oleh fungsi *callback* *OnDataSent* sebagai status gagal (`sendStatus != 0`).

i. **Pertanyaan:** Berapa detik waktu yang dibutuhkan oleh callback `changedConnectionCallback()` untuk mendeteksi bahwa salah satu simpulan telah keluar dari rantai mesh?

Hasil Pengamatan: Ketika tombol fisik `RST` ditekan atau daya salah satu board Node dipadamkan, *Serial Monitor* pada board yang masih menyala membutuhkan waktu sekitar **5 hingga 15 detik** sebelum fungsi `changedConnectionCallback()` terpanggil untuk mencatat perubahan topologi.

ii. **Pertanyaan:** Berapa lama waktu yang dibutuhkan hingga node tersebut kembali terdaftar secara otonom ke dalam topologi mesh tanpa konfigurasi ulang?

The screenshot displays the Arduino IDE interface. At the top, the title bar reads "sketch_oct6e | Arduino IDE 2.3.0". The menu bar includes "File", "Edit", "Sketch", "Tools", and "Help". Below the menu bar, there's a toolbar with icons for saving, opening files, and other IDE functions. The main editor area shows a C++ sketch named "sketch_oct6e.ino". The code defines a `newConnectionCallback` function that prints a message when a new node connects. It also sets up serial communication at 115200 baud, initializes a mesh network with a specific prefix, password, and scheduler, and adds a task to send messages. The bottom panel contains the "Output" window, which is currently showing the "Serial Monitor". The serial monitor displays the following output:

[Message [Enter to send message to 'NodeMCU 1.0 (ESP-12E Module)' on 'COM3']
New Line 115200 baud

16:51:32.642 -> Data Node 2 Terkirim: ("node":"Node-2","cahaya":2)
16:51:37.668 -> Data Node 2 Terkirim: ("node":"Node-2","cahaya":2)
16:51:42.643 -> Data Node 2 Terkirim: ("node":"Node-2","cahaya":2)
16:51:47.666 -> Data Node 2 Terkirim: ("node":"Node-2","cahaya":2)
16:51:49.092 -> r101DnCS0d0...
16:51:49.214 -> setLogLevel: ERROR | STARTUP |
16:51:49.214 -> STARTUP: init(): 0
16:51:49.306 -> STARTUP: AP tcp server established on port 5555
16:51:50.199 -> Data Node 2 Terkirim: ("node":"Node-2","cahaya":2)
16:51:55.188 -> Data Node 2 Terkirim: ("node":"Node-2","cahaya":2)

Analisis Teoretis: Hal ini membuktikan keunggulan arsitektur *Self-Organizing* dan *Self-Healing*. Node tidak memerlukan intervensi pengguna atau pengaturan ulang alamat MAC karena ia langsung mengenali parameter `MESH_PREFIX` dan `MESH_PASSWORD` yang sama secara otonom.

- i. Modifikasilah fungsi receivedCallback pada salah satu board agar bertindak sebagai **Pengawas Kondisi Suhu Kritis**: program **HANYA** mencetak kartu telemetri penuh ke Serial Monitor jika suhu yang dilaporkan oleh node tetangga bernilai **di atas 32.0 °C** (lakukan pengujian dengan mendekatkan jari hangat atau meniup sensor DHT pada node pengirim).
- ii. Jika suhu bernilai normal (≤ 32.0 °C), program cukup mencetak status singkat satu baris: [NORMAL] Telemetri dari Node <chipId> aman.
- iii. Tuliskan blok fungsi receivedCallback(uint32_t from, String &msg) hasil modifikasi Anda

The image shows the Arduino IDE interface with a sketch named "NodeMCU 1.0 (ESP-12E...)" open. The sketch code is as follows:

```
sketch_oct6e.ino
37 }
38
39 void newConnectionCallback(uint32_t nodeId) {
40   Serial.printf("Koneksi baru terdeteksi, nodeId: %u\n", nodeId);
41 }
42
43 void setup() {
44   Serial.begin(115200);
45
46   mesh.setDebugMsgTypes(ERROR | STARTUP);
47   mesh.init(MESH_PREFIX, MESH_PASSWORD, &userscheduler, MESH_PORT);
48   mesh.onReceive(&receivedCallback);
49   mesh.onNewConnection(&newConnectionCallback);
50
51   userscheduler.addTask(taskSendMessage);
52   taskSendMessage.enable();
53 }
54
55 void loop() {
56   mesh.update();
57 }
```

The Serial Monitor is open, showing the following output:

```
Ln 57, Col 2 NodeMCU 1.0 (ESP-12E Module) on COM3
Message (Enter to send message to 'NodeMCU 1.0 (ESP-12E Module)' on 'COM3')
16:51:32.642 -> Data Node 2 Terkirim: {"node":"Node-2","cahaya":2}
16:51:37.668 -> Data Node 2 Terkirim: {"node":"Node-2","cahaya":2}
16:51:42.643 -> Data Node 2 Terkirim: {"node":"Node-2","cahaya":2}
16:51:47.666 -> Data Node 2 Terkirim: {"node":"Node-2","cahaya":2}
16:51:49.092 -> r1D1QeQ8CmH...
16:51:49.214 -> setLogLevel: ERROR | STARTUP |
16:51:49.214 -> STARTUP: init(): 0
16:51:49.306 -> STARTUP: AP tcp server established on port 5555
16:51:50.199 -> Data Node 2 Terkirim: {"node":"Node-2","cahaya":2}
16:51:55.188 -> Data Node 2 Terkirim: {"node":"Node-2","cahaya":2}
```

blok fungsi receivedCallback(uint32_t from, String &msg)

```
void receivedCallback(uint32_t from, String &msg) {
    // Alokasi memori buffer JSON untuk memarsing pesan masuk
    StaticJsonDocument<200> jsonBuffer;
    DeserializationError error = deserializeJson(jsonBuffer, msg);

    // Abaikan jika format bukan JSON yang valid
    if (error) {
        return;
    }

    // Mengambil data suhu / cahaya dari pesan JSON
    float nilaiParameter = jsonBuffer["suhu"] | jsonBuffer["cahaya"] | 0.0;

    // Mengambil nama node pengirim
    const char* nodeName = jsonBuffer["node"] | "Unknown";

    // Filter kondisi kritis jika nilai di atas 32.0 atau di bawah 23.0
    if (nilaiParameter > 32.0 || nilaiParameter < 23.0) {
        Serial.println("=====");
        Serial.printf("[PERINGATAN] KONDISI KRITIS DARI NODE %s (%u)\n",
            nodeName, from);
        Serial.printf("Nilai Terukur: %.2f\n", nilaiParameter);
        Serial.println("Detail Telemetri: " + msg);
        Serial.println("=====");
    } else {
        // Jika nilai normal (antara 23.0 sampai 32.0)
        Serial.printf("[NORMAL] Telemetri dari Node %s aman (Nilai: %.2f).\n",
            nodeName, nilaiParameter);
    }
}
```